

TECNOLOGÍA EN SISTEMAS Y GESTIÓN DE DATOS

Desarrollo de un Sistema Distribuido de Gestión de Productos utilizando SOAP, WSDL y Node.js

Informe técnico — Tarea práctica de Aplicaciones Distribuidas

Integrantes

Ismael Vanegas
Alexandra Caicedo
Israel Arevalo

Asignatura: Aplicaciones Distribuidas

Docente: [Nombre del docente]

Agosto de 2026

1. Introducción

Cuando una aplicación deja de vivir en una sola máquina y empieza a repartir su trabajo entre programas distintos que se comunican entre sí —estén en la misma computadora o en extremos opuestos del mundo—, hablamos de una aplicación distribuida. Esa arquitectura es, en la práctica, la forma en que funciona casi todo lo que usamos a diario: cuando alguien consulta el clima en su celular o revisa su banco en línea, hay un cliente pidiendo información y un servidor respondiéndola, casi siempre sin que el usuario note la separación.

Para que esa conversación entre cliente y servidor sea posible, ambos deben hablar el mismo idioma. SOAP (Simple Object Access Protocol) es uno de esos idiomas: un protocolo basado en XML que define reglas estrictas para empaquetar solicitudes y respuestas, apoyado en un documento llamado WSDL (Web Services Description Language) que describe, de forma exacta, qué operaciones ofrece un servicio, qué datos espera recibir y qué va a devolver. Es, en cierto sentido, un contrato: mientras el cliente y el servidor lo respeten, no importa en qué lenguaje de programación esté escrito cada uno.

Este informe documenta el desarrollo de un sistema de gestión de productos construido sobre ese modelo: un servidor SOAP hecho en Node.js, con su propio archivo WSDL, capaz de registrar, consultar, listar, actualizar, valorizar y eliminar productos. Para comprobar que el contrato funciona sin importar el lenguaje del cliente, lo consumimos desde tres clientes completamente distintos —Python, PHP y C#—, cada uno con su propio panel web además de su cliente de consola. Y, ya entendiendo bien el protocolo, decidimos ir más allá de lo mínimo exigido: le agregamos persistencia en una base de datos (Supabase), una API REST sobre el mismo backend para comparar ambos protocolos en la práctica, y una mejora adicional más ambiciosa: gestión de locales (sucursales) con ubicación geográfica, stock por tienda y un mapa interactivo, expuesta también por SOAP mediante un segundo servicio dentro del mismo WSDL. Todo lo que se describe a continuación fue construido, ejecutado y probado por el equipo; no es un ejercicio únicamente teórico.

El trabajo se dividió así: Ismael desarrolló el servidor SOAP/REST, el front de PHP, y diseñó e implementó la mejora adicional de locales/mapa/stock por tienda en los tres fronts. Alexandra construyó el cliente de consola en C# con su mapa interactivo —el aporte individual sobre el que después se levantó el front web de ese mismo lenguaje— y ejecutó las validaciones y pruebas en SoapUI. Israel desarrolló el cliente y el front web de Python, y realizó una revisión general de todo el sistema (servidor, los tres fronts y la mejora adicional) antes de la entrega final.

2. Objetivo general

Desarrollar una aplicación distribuida cliente-servidor que implemente un servicio web SOAP en Node.js, utilizando un documento WSDL para definir las operaciones, los parámetros de entrada y las respuestas del servicio de gestión de productos.

3. Objetivos específicos

- Comprender el funcionamiento del protocolo SOAP y la estructura de un documento WSDL, no solo en teoría sino construyendo uno desde cero y extendiéndolo después con un segundo servicio.
- Implementar un servidor SOAP en Node.js que exponga las seis operaciones de gestión de productos exigidas por la tarea, con sus respectivas validaciones de negocio.
- Desarrollar clientes en tres lenguajes de programación distintos —Python, PHP y C#— capaces de consumir el mismo servicio sin modificarlo, cada uno con su propio panel web.
- Intercambiar información estructurada mediante mensajes XML y comprobar, en la práctica, que el contrato definido en el WSDL se respeta en todas direcciones.
- Probar cada una de las operaciones del servicio utilizando SoapUI, cubriendo tanto casos correctos como casos de error.
- Documentar todo el proceso de creación, ejecución y validación de la aplicación distribuida, incluyendo las decisiones técnicas tomadas en el camino.

4. Marco teórico

4.1 Aplicaciones distribuidas

Una aplicación distribuida es un sistema de software cuyos componentes se ejecutan en procesos separados —muchas veces en computadoras distintas— y que coordinan su trabajo enviándose mensajes a través de una red, en lugar de compartir memoria directamente. La ventaja de este enfoque es que cada componente puede desarrollarse, desplegarse y escalarse de forma independiente: en nuestro proyecto, por ejemplo, el servidor Node.js no necesita saber en qué lenguaje está escrito el cliente que lo consulta, y cada uno de los tres integrantes puede levantar su propia copia del servidor sin que eso afecte a los demás.

4.2 Servicios web

Un servicio web es una forma estandarizada de exponer funcionalidad de un sistema para que otros programas la consuman a través de la red, generalmente sobre HTTP. En nuestro caso, el mismo servidor Node.js expone varios servicios sobre el mismo puerto: dos servicios SOAP —ProductosService, con las seis operaciones calificadas, y LocalesService, la mejora adicional de locales y mapa— y, además, una API REST (en las rutas `/api/*`). Todos terminan ejecutando la misma lógica de negocio de fondo, lo cual nos permitió comparar en la práctica cómo cada protocolo empaqueta y transporta la misma información, y también cómo un mismo documento WSDL puede describir más de un servicio a la vez.

4.3 Protocolo SOAP

SOAP es un protocolo de mensajería basado en XML que define un formato estricto para el intercambio de información entre aplicaciones: cada mensaje viaja dentro de un “sobre” (Envelope) que contiene un encabezado opcional y un cuerpo (Body) con la solicitud o la respuesta. A diferencia de otros estilos de comunicación más flexibles, SOAP exige que la estructura de cada mensaje coincida exactamente con lo que el WSDL declaró; si el servidor incluye un dato que el contrato no definió, el mensaje deja de ser válido. Esto lo aprendimos de la peor manera: al agregar un campo adicional para identificar qué integrante había registrado cada producto, la librería que usamos terminó incluyéndolo también en las respuestas SOAP, rompiendo la conformidad del contrato. Tuvimos que separar explícitamente esa información para que solo viajara por la API REST, y dejar el SOAP tal como el WSDL lo describe.

4.4 Lenguaje XML

XML (Extensible Markup Language) es un lenguaje de marcado diseñado para representar información estructurada de forma legible tanto para personas como para programas, usando etiquetas que el propio autor del documento define según lo que necesite describir. SOAP se apoya completamente en XML: cada solicitud que enviamos desde SoapUI —por ejemplo, para registrar un producto— y cada respuesta que el servidor devuelve, es un documento XML válido, con las etiquetas <codigo>, <nombre>, <precio>, etc., anidadas dentro del sobre SOAP.

4.5 Archivo WSDL

El WSDL es el documento que describe formalmente uno o más servicios SOAP: qué operaciones ofrece cada uno, qué tipos de datos usa (types), qué mensajes de entrada y salida corresponden a cada operación (message y portType), cómo se transportan esos mensajes (binding) y en qué dirección de red se puede acceder a cada servicio (service y port). En este proyecto escribimos el archivo productos.wsdl a mano, definiendo primero las seis operaciones exigidas por el enunciado dentro de ProductosPortType, y lo dejamos accesible desde el navegador en <http://localhost:8000/productos?wsdl>, tal como pide la tarea. Más adelante, para la mejora adicional de locales, aprendimos que un mismo documento WSDL puede declarar más de un service —cada uno con su propio portType, binding y dirección de red— sin alterar en absoluto los que ya existían: así agregamos LocalesPortType sin tocar ni una línea del contrato original. Ese mismo archivo es el que SoapUI necesita para generar automáticamente las solicitudes de prueba, y el que las librerías cliente (zeep en Python, SoapClient en PHP, y el proxy generado por dotnet-svcutil en C#) leen para saber qué operaciones pueden invocar.

4.6 Arquitectura cliente-servidor

La arquitectura cliente-servidor divide un sistema en dos roles: el servidor, que concentra los datos y la lógica de negocio y espera solicitudes, y el cliente, que las envía y presenta los resultados. En nuestro proyecto el servidor es único (Node.js, con su lista de productos en memoria), pero tiene múltiples clientes que no se conocen entre sí: tres clientes de consola (Python, PHP y C#) que demuestran las seis operaciones exigidas por la tarea, y tres paneles web —uno por lenguaje, cada uno con su propio color de

acento— que además incorporan la mejora adicional de locales y mapa interactivo. Ninguno de los clientes necesita saber cómo está implementado el servidor por dentro; les basta con el contrato que describen los dos servicios del WSDL.

4.7 SOAP frente a REST

REST (Representational State Transfer) es un estilo arquitectónico —no un protocolo con una especificación única como SOAP— que aprovecha directamente los verbos de HTTP (GET, POST, PATCH, DELETE) para representar acciones sobre recursos, normalmente intercambiando datos en JSON. SOAP, en cambio, siempre usa XML, define su propio contrato formal (el WSDL) y es más rígido en cuanto a validación y tipado. Al construir ambas versiones sobre el mismo backend —SOAP en `/productos` y REST en `/api/productos`, compartiendo la misma lógica de validación en `operaciones.js`— pudimos comparar directamente: la versión REST resultó más liviana de invocar y más fácil de consumir desde herramientas como Postman, mientras que la versión SOAP, aunque más verbosa, ofrece un contrato explícito y autodescriptivo que SoapUI puede leer sin que nosotros documentemos nada aparte. Esta comparación se volvió más concreta todavía cuando decidimos exponer también la mejora de locales por SOAP en vez de dejarla solo en REST: aunque REST hubiera sido más rápido de implementar, hacerlo por SOAP fue justamente el ejercicio que más nos ayudó a entender en la práctica —no solo en la teoría— qué significa “extender un contrato” sin romperlo.

5. Arquitectura de la aplicación

El diagrama siguiente resume cómo está organizado el sistema compartido por el equipo. Cada integrante corre su propia instancia local del servidor Node.js (puerto 8000), que expone dos servicios SOAP (`ProductosService` y `LocalesService`) y una API REST; las tres instancias comparten una única base de datos en Supabase, que actúa como persistencia adicional sobre la lista en memoria de cada servidor y como almacén de los locales/sucursales.

Arquitectura del sistema — Proyecto SOAP Productos

Tres instancias locales, un mismo contrato SOAP/REST, una base de datos compartida

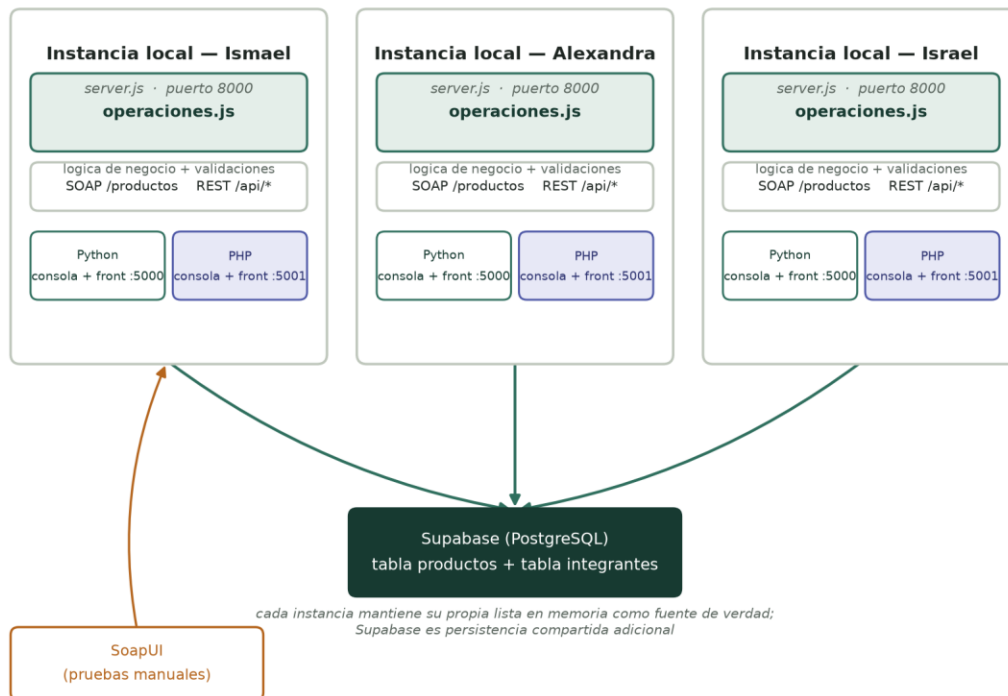


Figura 1. Arquitectura general del sistema: tres instancias locales, dos servicios SOAP sobre un mismo WSDL, una base de datos compartida.

La pieza central del servidor es `operaciones.js`: un módulo que concentra toda la lógica de negocio y las validaciones de las seis operaciones calificadas. Tanto el servicio SOAP (`server.js`) como la API REST (`rest.js`) son, en el fondo, envoltorios delgados sobre ese mismo módulo — así garantizamos que, sin importar por cuál de los dos protocolos entre una solicitud, la respuesta ante los mismos datos sea idéntica. La mejora adicional de locales sigue el mismo patrón, en `operacionesLocales.js`, compartido a su vez entre `LocalesService` (SOAP) y las rutas REST equivalentes. Por debajo, `data/productos.js` mantiene el arreglo en memoria que es la fuente de verdad operativa (tal como exige el enunciado), mientras que `db/supabaseClient.js` intenta reflejar cada cambio en Supabase de forma adicional; si Supabase no responde, el servidor sigue funcionando exclusivamente en memoria para las seis operaciones obligatorias, sin caerse. La gestión de locales sí depende de Supabase (una ubicación geográfica no tiene sentido como dato puramente en memoria), y lo indica explícitamente si la base no está disponible.

Sobre ese mismo servidor se apoyan seis clientes: tres de consola (`cliente-python/client.py` con la librería `zeep`, `cliente-php/client.php` con la clase nativa `SoapClient`, y `cliente-csharp/Program.cs` con un proxy generado por `dotnet-svcutil`) que demuestran las operaciones exigidas por el enunciado, y tres paneles web —uno por lenguaje— que construimos como mejora adicional, cada uno con su propio color de

acento (verde azulado en Python, índigo en PHP, naranja en C#) y los siete apartados del menú lateral: las seis operaciones calificadas, más “Locales y stock”.

5.1 Roles del equipo

- Ismael — servidor SOAP + REST, front web de PHP, y diseño e implementación de la mejora adicional de locales/mapa/stock por tienda (incluyendo el segundo servicio SOAP) en los tres fronts.
- Alexandra — cliente de consola en C# con mapa interactivo (idea y desarrollo original, aporte individual); base sobre la que después se construyó el front web de ese mismo lenguaje. También ejecutó las validaciones y pruebas en SoapUI.
- Israel — cliente y front web de Python; revisión general (check) de todo el sistema —servidor, los tres fronts y la mejora adicional— antes de la entrega final.

5.2 URLs del sistema

Con el servidor y los tres fronts corriendo en local:

- WSDL: <http://localhost:8000/productos?wsdl>
- SOAP — ProductosService (6 operaciones calificadas): <http://localhost:8000/productos>
- SOAP — LocalesService (mejora adicional): <http://localhost:8000/locales>
- API REST (información general): <http://localhost:8000/>
- Front Python: <http://localhost:5000>
- Front PHP: <http://localhost:5001>
- Front C#: <http://localhost:5002>

6. Desarrollo

6.1 Instalación de Node.js y creación del proyecto

El punto de partida fue instalar Node.js y crear la carpeta del proyecto siguiendo la estructura sugerida por el enunciado (servidor/, cliente-python/, cliente-php/, evidencias/, informe/). Dentro de servidor/ inicializamos el proyecto con npm y agregamos las dependencias necesarias:

```
npm install express soap
```

6.2 Diseño del archivo WSDL

Antes de escribir una sola línea del servidor, definimos el contrato: creamos productos.wsdl declarando el tipo complejo Producto (código, nombre, categoría, precio, cantidad) y, para cada una de las seis operaciones, su mensaje de entrada, su mensaje de salida, la entrada en el portType, y su asociación al binding SOAP sobre HTTP. Al probarlo por primera vez con node-soap nos topamos con un error de

“comentario mal formado”: habíamos usado líneas decorativas con guiones dobles (--) dentro de comentarios XML, algo que la especificación XML prohíbe explícitamente. Corregirlo nos obligó a entender que un WSDL, al final, es un documento XML como cualquier otro y hereda todas sus reglas.

6.3 Implementación del servidor

Con el contrato definido, implementamos server.js utilizando la librería soap sobre un servidor Express, montando el servicio en la ruta /productos y dejando el WSDL disponible en /productos?wsdl. Cada una de las seis operaciones quedó implementada validando primero los datos de entrada (código no vacío, no duplicado, nombre y categoría no vacíos, precio mayor que cero, cantidad entera igual o mayor que cero, y existencia del producto cuando aplica), y devolviendo siempre un objeto con los campos estado y mensaje, tal como pide el enunciado. Cada operación ejecutada queda registrada en la consola del servidor con hora y resultado, lo que facilitó muchísimo depurar errores durante las pruebas.

6.4 Persistencia adicional con Supabase

Como mejora adicional, conectamos el servidor a una base de datos Postgres administrada por Supabase. El diseño es híbrido a propósito: el arreglo en memoria sigue siendo la fuente de verdad para responder cada solicitud de las seis operaciones obligatorias (así el servidor jamás depende de que Supabase esté disponible para cumplir el requisito de la tarea), pero cada operación que modifica datos también intenta reflejar el cambio en la base compartida. Como los tres integrantes corremos nuestra propia instancia local apuntando a la misma base, agregamos una columna origen que registra qué instancia creó cada producto, y un endpoint adicional (GET /api/instancia) que cada panel web consulta para mostrar con qué servidor está hablando en ese momento.

6.5 Implementación de los clientes de consola

Desarrollamos tres clientes de consola: client.py en Python, usando la librería zeep, que lee el WSDL y genera dinámicamente las funciones correspondientes a cada operación; client.php en PHP, usando la clase nativa SoapClient, sin ninguna librería externa; y Program.cs en C#, usando un proxy generado con la herramienta dotnet-svcutil a partir del mismo WSDL. Los tres ejecutan la misma secuencia de pruebas: registran dos productos, consultan uno existente y uno inexistente, listan el inventario, actualizan el stock de un producto, calculan el valor de su inventario y finalmente lo eliminan —incluyendo, en cada paso relevante, un intento con datos inválidos para comprobar que el servidor rechaza correctamente el caso incorrecto.

6.6 Cliente en C# y su front web

El cliente en C# nació como aporte individual de Alexandra: un cliente de consola (.NET) que, además de las 7 demostraciones mínimas, genera un mapa interactivo con la librería Leaflet —cada categoría de producto ubicada en una bodega o tienda real (Quito, Guayaquil, Cuenca, Ambato), abierto

automáticamente en el navegador como un archivo HTML independiente (mapa_productos.html). Una decisión de diseño importante desde el inicio: ese mapeo categoría-bodega vive únicamente dentro del cliente C# (en MapaBodegas.cs), sin agregar ningún campo nuevo al WSDL ni a la tabla productos de Supabase — infraestructura compartida por los tres integrantes, que una mejora individual no debía modificar.

Sobre esa misma base, Ismael extendió la idea de Alexandra a un panel web completo (cliente-csharp/front/, ASP.NET Core Razor Pages), replicando exactamente el mismo patrón ya construido para Python y PHP: las seis operaciones en el mismo orden, la mejora de locales/mapa, y un color de acento propio (naranja) para distinguirlo a simple vista de los otros dos. Técnicamente, el front reutiliza el mismo proxy SOAP generado por dotnet-svcutil que ya usaba el cliente de consola de Alexandra (no se regeneró nada aparte); solo se le agregó el cliente correspondiente a LocalesService cuando se construyó la mejora adicional. Así, C# terminó siendo un lenguaje más con paridad completa frente a Python y PHP —cliente de consola y panel web—, no un aporte aparte.

6.7 Mejora adicional: locales, mapa y stock por tienda (SOAP)

La idea surgió de una pregunta simple: si un producto tiene una cantidad total, ¿en qué tienda física está realmente ese stock? A partir de ahí diseñamos una gestión de locales (sucursales) con nombre y ubicación geográfica, la posibilidad de asignar cuánto stock de cada producto hay en cada local, un mapa interactivo (Leaflet) para verlo de un vistazo, y un cálculo de valor de inventario por tienda —análogo a CalcularValorInventario, pero acotado a una sola sucursal.

La primera versión de esta mejora se construyó solo por REST, siguiendo el mismo criterio que ya usábamos para el equipo y la instancia (datos de apoyo, no parte del contrato calificado). Pero, revisando el enunciado con más calma, entendimos que el espíritu de la tarea es precisamente trabajar con SOAP y WSDL — dejar una mejora de este tamaño únicamente en REST desaprovechaba justo lo que se supone que debíamos practicar. Rehicimos la mejora para que pasara por SOAP, con una decisión de diseño concreta: en vez de mezclar las nuevas operaciones dentro de ProductosPortType (lo que habría dejado un contrato calificado de seis operaciones con catorce operaciones mezcladas, difícil de distinguir para quien lo revise), agregamos un service completamente nuevo, LocalesService, con su propio portType (LocalesPortType), su propio binding y su propia dirección de red — todo dentro del mismo archivo productos.wsdl. Las seis operaciones originales no cambiaron ni un carácter.

Al probar esta segunda versión nos encontramos con un problema real de la librería node-soap: internamente, el servidor decide a qué binding pertenece una solicitud entrante comparando la ruta HTTP de la petición contra la dirección declarada en el WSDL para cada port — y si dos service declaran la misma dirección, siempre resuelve al primero, sin importar qué operación se pidió en realidad. La primera vez que probamos ListarLocales, el servidor intentó procesarla como si fuera una operación de ProductosPortType y falló. La solución fue darle a LocalesPort su propia dirección de red

(<http://localhost:8000/locales>, distinta de [/productos](#)), y escuchar en esa ruta también desde `server.js`. Fue, en la práctica, la lección más concreta de todo el proyecto sobre qué significa que un service y un port definan “dónde vive” un servicio SOAP — algo que en la teoría del WSDL suena simple, pero que solo se entiende del todo cuando una librería real se comporta de forma inesperada.

Del lado del servidor, la lógica de negocio vive en `operacionesLocales.js` —mismo patrón que `operaciones.js`—, compartida entre el servicio SOAP y los endpoints REST equivalentes (estos últimos se conservaron solo como comodidad para probar con Postman, tal como sugiere el enunciado, pero ningún front los usa ya). Del lado de los tres clientes, cada lenguaje resolvió la conexión al segundo servicio de forma distinta y esa diferencia terminó siendo, en sí misma, un aprendizaje: en Python, `zeep` exige enlazar explícitamente el servicio y el puerto (`client.bind("LocalesService", "LocalesPort")`), porque no asume cuál de los dos usar cuando hay más de uno; en PHP, `SoapClient` fusiona automáticamente las operaciones de ambos servicios en un mismo cliente y enruta cada llamada a la dirección correcta sin configuración adicional; en C#, `dotnet-svcutil` generó un cliente independiente por servicio (`ProductosPortTypeClient` y `LocalesPortTypeClient`) a partir del mismo WSDL, sin tocar ni una línea del primero — confirmando, otra vez, que agregar un service nuevo es un cambio aditivo y seguro.

6.8 API REST y paneles web

Para poner en práctica la comparación entre SOAP y REST, expusimos las mismas seis operaciones también como una API REST (GET, POST, PATCH y DELETE sobre [/api/productos](#)), reutilizando la misma lógica de validación del servidor SOAP. Sobre esa base construimos tres paneles web —Flask para Python, PHP puro sin framework, y ASP.NET Core Razor Pages para C#— con un menú lateral que organiza las seis operaciones en el mismo orden del enunciado más un séptimo apartado para la mejora de locales, cada uno con su propio color de acento para distinguirlos a simple vista, y los tres mostrando en tiempo real con qué instancia del servidor están hablando y qué integrante registró cada producto.

6.9 Pruebas

Antes de dar por cerrado el desarrollo, importamos `productos.wsdl` en SoapUI y ejecutamos al menos un caso correcto y un caso incorrecto por cada una de las seis operaciones calificadas (trece casos en total, incluyendo un caso estructural de SOAP Fault sobre `ListarProductos`, que no admite parámetros y por tanto no tiene un caso incorrecto de negocio propio). Las capturas de estas pruebas se documentan en la sección de evidencias. Las ocho operaciones de `LocalesService`, al ser una mejora adicional fuera del alcance obligatorio de SoapUI, se verificaron con un cliente SOAP de prueba propio (un script de Node.js con la misma librería `soap`) y, de forma interactiva, navegando cada uno de los tres paneles web de punta a punta.

7. Evidencias

Esta sección reúne las capturas que demuestran el funcionamiento real del sistema. El conjunto completo de evidencias de SoapUI (14 archivos) está disponible en la carpeta evidencias/ del repositorio; aquí se incluye una muestra representativa, más las capturas nuevas de la mejora adicional.

7.1 WSDL importado en SoapUI

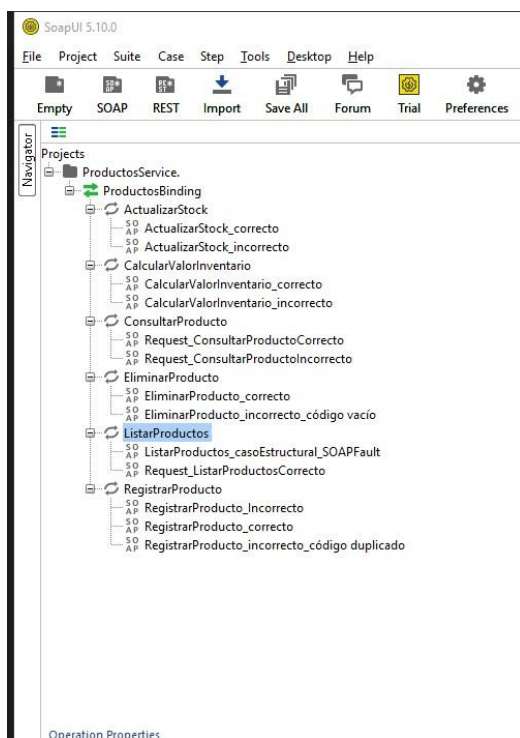


Figura 2. Árbol de operaciones generado por SoapUI al importar productos.wsdl — las seis operaciones calificadas quedan disponibles, cada una con su caso correcto e incorrecto.

7.2 Caso correcto — RegistrarProducto

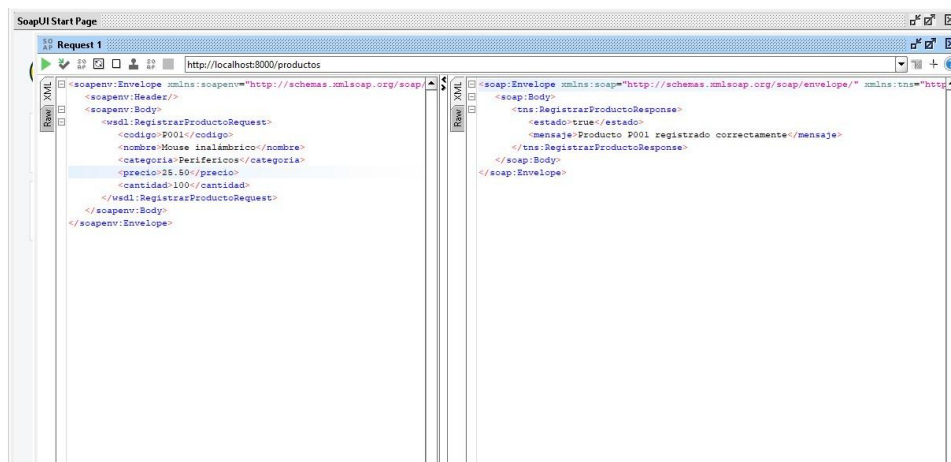


Figura 3. Solicitud y respuesta SOAP de un registro exitoso (estado = true).

7.3 Caso incorrecto — código duplicado

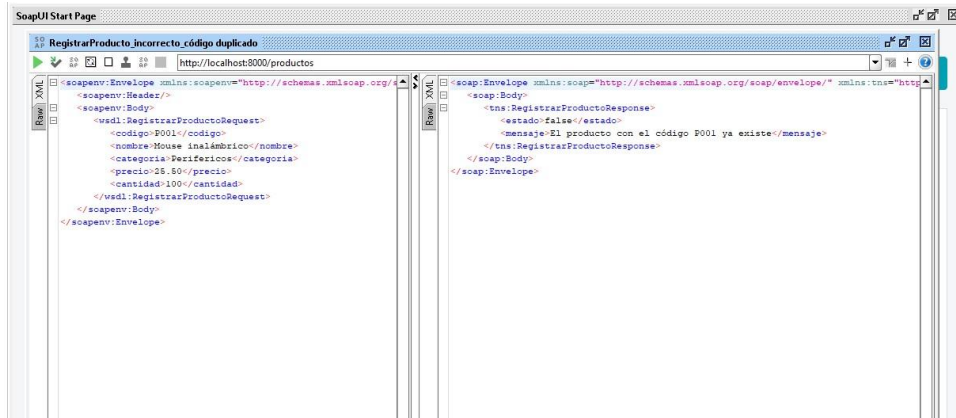


Figura 4. El servidor rechaza un código ya existente, devolviendo estado = false y un mensaje explicando el motivo.

7.4 Caso incorrecto — producto inexistente

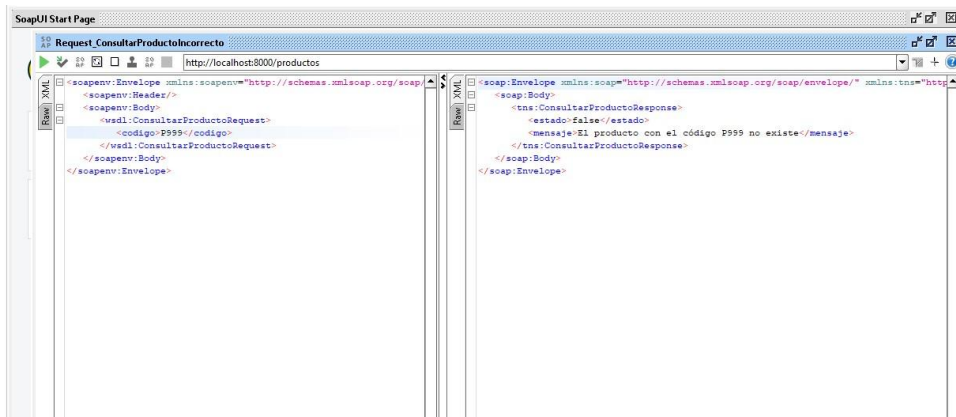


Figura 5. ConsultarProducto ante un código que no existe.

7.5 Caso estructural — SOAP Fault

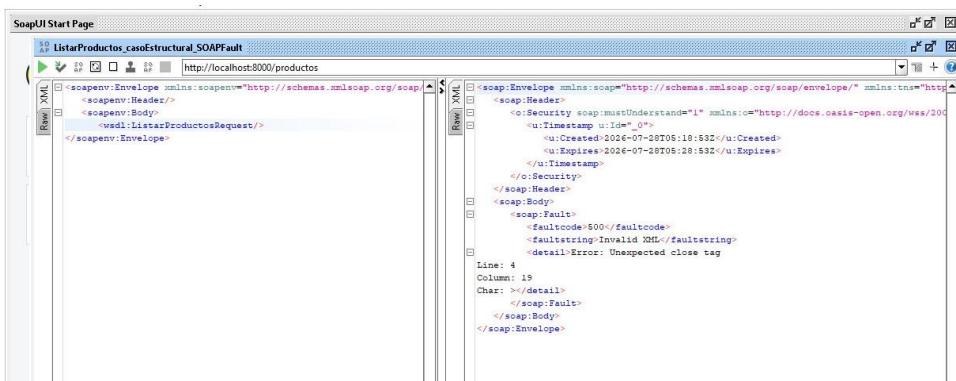


Figura 6. Como ListarProductos no recibe parámetros, se documentó en su lugar un XML mal formado para mostrar cómo el motor SOAP responde con un Fault ante una solicitud inválida en su estructura.

7.6 Caso correcto — EliminarProducto

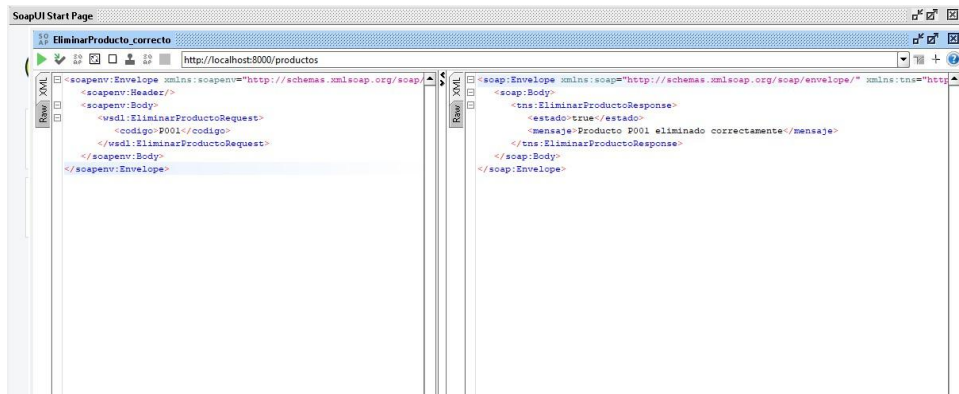


Figura 7. Eliminación exitosa de un producto.

7.7 Ejecución del servidor

El servidor se levanta con `npm start` desde la carpeta `servidor/` y muestra en consola la hidratación desde Supabase, la dirección del servicio SOAP y de la API REST, y cada operación ejecutada con su resultado.

```

PS D:\Ismael\DESARROLLO\aplicaciones_distribuidas\proyecto-soap-productos\servidor> npm start

> servidor-soap-productos@1.0.0 start
> node server.js

Hidratando memoria desde Supabase (si está configurado)...
Supabase conectado. 3 producto(s) cargado(s) en memoria.
Servidor SOAP escuchando en http://localhost:8000/productos
WSDL disponible en http://localhost:8000/productos?wsdl
API REST disponible en http://localhost:8000/api/productos
  
```

Figura 8. Consola del servidor tras "npm start": WSDL y API REST disponibles.

7.8 WSDL con los dos servicios (ProductosService y LocalesService)

`http://localhost:8000/productos?wsdl`

CAPTURA PENDIENTE

Captura del WSDL en el navegador (Ctrl+F "service name") o del árbol de SoapUI, mostrando ambos
<service>: ProductosService y LocalesService.

Archivo sugerido: evidencias/14_WSDL_dos_servicios.png

Figura 9. El mismo archivo WSDL describe dos servicios SOAP independientes.

7.9 Paneles web — Python, PHP y C#

Los tres paneles consumen el mismo WSDL y muestran, en su barra lateral, con qué instancia del servidor están hablando y quién registró cada producto (columna Origen), demostrando el uso conjunto de SOAP (para las operaciones) y REST (para el equipo y la instancia).

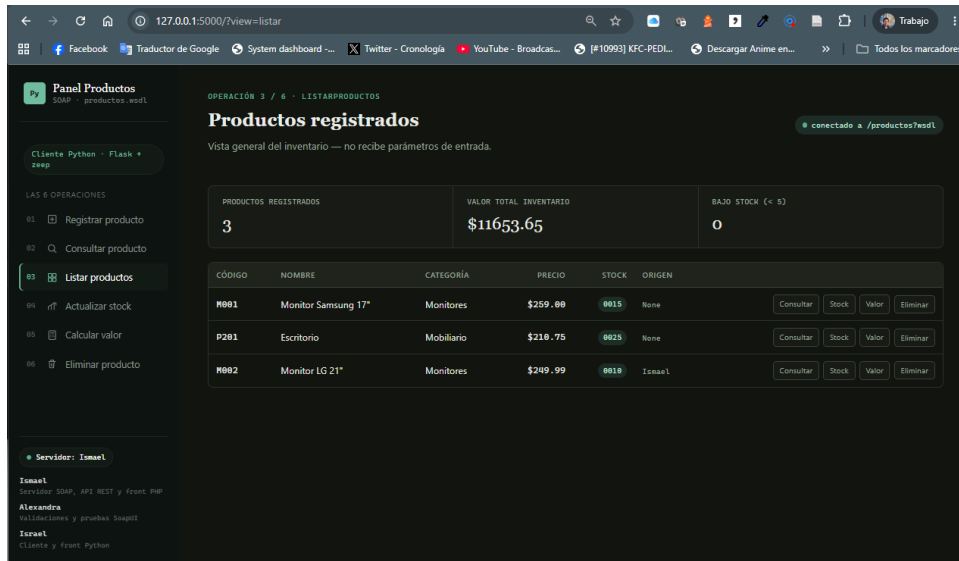


Figura 10. Panel web del cliente Python (Flask + zeep).

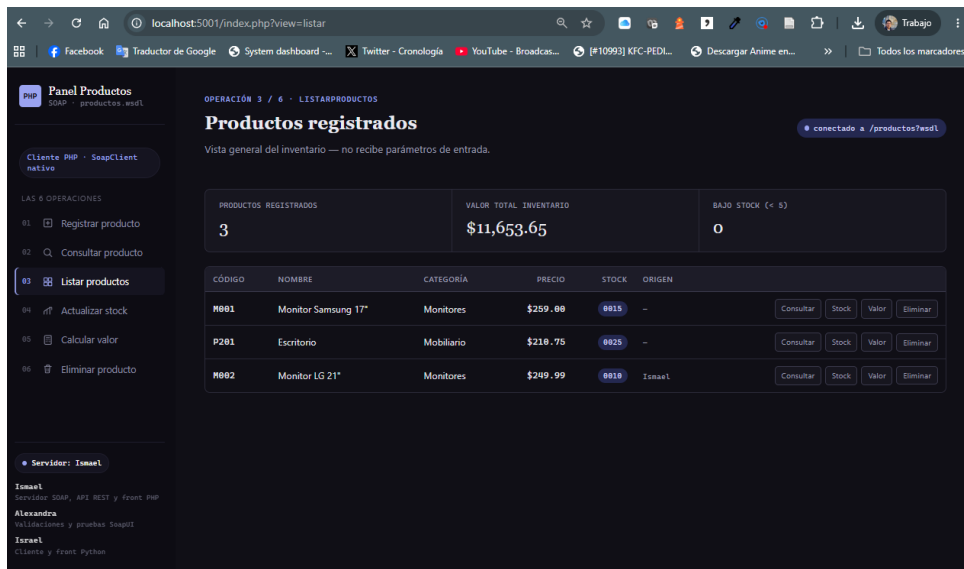


Figura 11. Panel web del cliente PHP (SoapClient nativo).

CAPTURA PENDIENTE

Captura del front C# (http://localhost:5002), barra lateral con el badge “Cliente C# · ASP.NET Core + dotnet-svcutil” y su acento naranja.

Archivo sugerido: evidencias/15_front_csharp.png

Figura 12. Panel web del cliente C# (ASP.NET Core Razor Pages).

7.10 Mapa interactivo del cliente de consola en C# (Leaflet)

El cliente de consola en C# incorpora un mapa hecho con la librería Leaflet: al ejecutarlo (con "dotnet run" o, sin registrar ni borrar nada, con "dotnet run -- --mapa") se genera un archivo mapa_productos.html que

se abre automáticamente en el navegador, ubicando cada producto en la bodega o tienda que corresponde a su categoría. Esta evidencia es parte del aporte individual original de Alexandra.

CAPTURA PENDIENTE

Captura del archivo `mapa_productos.html` abierto en el navegador tras correr “`dotnet run -- --mapa`” en `cliente-csharp/`.

Archivo sugerido: evidencias/16_mapa_consola_csharp.png

7.11 Mejora adicional: locales, mapa y stock por tienda

La pestaña “Locales y stock” (07) existe igual en los tres fronts: un formulario para crear/editar/eliminar tiendas, y un mapa central que, al hacer clic en cada marcador, muestra qué productos y cuánto stock tiene esa tienda en particular, junto con su valor de inventario.

CAPTURA PENDIENTE

Captura de la pestaña “Locales y stock” en cualquiera de los tres fronts, con el mapa mostrando varias tiendas y el popup de una de ellas abierto (nombre, productos, valor total).

Archivo sugerido: evidencias/17_locales_mapa.png

Figura 13. Mapa de tiendas con productos y valor de inventario por sucursal.

CAPTURA PENDIENTE

Captura de la pantalla “Registrar producto” mostrando el selector opcional de “Tienda inicial”, y el mensaje de confirmación tras registrar (`RegistrarProducto + AsignarStockLocal` encadenados).

Archivo sugerido: evidencias/18_registrar_con_tienda.png

Figura 14. Registro de un producto con asignación de tienda inicial en el mismo formulario.

8. Análisis de resultados

El servicio SOAP respondió de forma consistente a lo largo de todas las pruebas: las seis operaciones calificadas funcionan tanto desde SoapUI como desde los tres clientes de consola y los tres paneles web, y las validaciones de negocio se comportan igual sin importar por cuál de las dos vías —SOAP o REST— llegue la solicitud, gracias a que ambas comparten la misma capa de lógica (`operaciones.js`). Esto nos permitió confirmar en la práctica algo que solo habíamos leído en la teoría: un buen contrato de servicio (ya sea un WSDL o una API REST bien diseñada) permite que clientes completamente distintos —de consola o web, en Python, PHP o C#— interactúen con el mismo backend sin conocer los detalles de su implementación.

La comunicación cliente-servidor se dio siempre sobre HTTP, con el cuerpo de cada mensaje en XML para el canal SOAP y en JSON para el canal REST. El primer inconveniente real que enfrentamos, ya mencionado

en el marco teórico, fue que al agregar el campo origen para identificar la instancia que registraba cada producto, la librería node-soap lo serializó también en las respuestas SOAP a pesar de no estar declarado en el WSDL, rompiendo la conformidad estricta del contrato. Detectarlo nos obligó a inspeccionar el XML crudo de la respuesta —no solo la versión ya interpretada por el cliente— y a separar explícitamente qué datos viajan por cada protocolo.

El segundo inconveniente, ya descrito en la sección 6.7, apareció al extender el WSDL con LocalesService: node-soap resolvía el binding de cada solicitud entrante únicamente por la ruta HTTP, no por la operación pedida, así que mientras los dos service compartieron la misma dirección, todo lo que llegaba se procesaba como si fuera de ProductosPortType. Fue un error silencioso hasta que probamos la primera operación nueva (ListarLocales) y el servidor devolvió un fallo interno en vez de la lista esperada. Corregirlo —dándole a cada servicio su propia dirección de red— nos dejó una idea mucho más concreta de qué es, en la práctica, un port dentro de un WSDL: no es solo una etiqueta descriptiva, es información que la librería SOAP usa activamente para enrutar.

Trabajar con tres integrantes corriendo cada uno su propio servidor local, apuntando a la misma base de datos compartida en Supabase, también puso a prueba un supuesto que dábamos por sentado: que “nuestros datos” eran solo nuestros. Más de una vez, al listar productos, aparecían registros de prueba de otro integrante. Resolverlo (con la columna origen y el identificador de instancia) terminó siendo, sin haberlo planeado, una lección adicional sobre los problemas reales de compartir estado entre sistemas distribuidos.

9. Conclusiones

- SOAP resultó ser, tal como plantea la teoría, un protocolo más rígido y verboso que REST, pero esa misma rigidez es la que permite que herramientas como SoapUI generen automáticamente solicitudes de prueba a partir del WSDL, sin que el equipo tenga que documentar manualmente cada operación aparte.
- Un WSDL bien definido es, en la práctica, más que documentación: es el contrato que hace posible que clientes en Python, PHP y C# —que no comparten ni una línea de código entre sí— consuman exactamente el mismo servicio sin errores de interpretación, incluso cuando ese WSDL crece para incluir un segundo servicio.
- Un mismo documento WSDL puede alojar más de un servicio SOAP (service/portType/binding independientes) sin alterar el contrato calificado ya existente. Elegir ese camino para la mejora adicional de locales, en vez de dejarla solo en REST, fue lo que más nos ayudó a entender la diferencia real entre SOAP y REST —no la que se lee en un resumen teórico, sino la que se siente al depurar por qué una operación nueva no llega a donde debería.
- Separar la lógica de negocio (operaciones.js, operacionesLocales.js) de la capa de transporte (SOAP en server.js, REST en rest.js) resultó clave para mantener ambos protocolos consistentes entre sí, y

facilitó detectar y corregir los dos problemas de serialización/enrutamiento que describimos en el análisis de resultados.

- Diseñar el servidor para que la persistencia en base de datos sea un complemento y no una dependencia (memoria como fuente de verdad para las seis operaciones calificadas, Supabase como mejora adicional) demostró ser una decisión acertada: nunca perdimos funcionalidad por una caída o mala configuración de la base de datos.
- Ir más allá del mínimo exigido —agregando la API REST, tres paneles web completos y la gestión de locales con mapa interactivo por SOAP— terminó siendo la parte que más nos ayudó a entender las diferencias reales entre SOAP y REST, más que cualquier lectura teórica previa.

10. Recomendaciones

- Antes de agregar cualquier campo nuevo a una respuesta SOAP, revisar el XML crudo generado (no solo la versión ya parseada por el cliente) para confirmar que sigue siendo válido contra el WSDL declarado.
- Si un WSDL va a crecer para incluir más de un servicio, probar cada operación nueva con un cliente SOAP real (no solo revisar el XML) apenas se agregue, en vez de esperar a integrarla en los clientes finales — así se detectan a tiempo comportamientos de enrutamiento específicos de la librería, como el que documentamos en la sección 6.7.
- Si un equipo va a compartir una misma base de datos entre varias instancias locales del mismo servidor, definir desde el inicio algún identificador de origen — evita confusión y facilita depurar cuando los datos de prueba de distintos integrantes se mezclan.
- Cuando un integrante aporta una idea individual que luego el equipo decide extender (como el cliente en C#), mantener el reconocimiento de esa autoría original en la documentación, incluso después de que el resto del equipo construya sobre ella.

11. Bibliografía

Bray, T., Paoli, J., Sperberg-McQueen, C. M., Maler, E., & Yergeau, F. (2008). Extensible Markup Language (XML) 1.0 (5.^a ed.). World Wide Web Consortium. <https://www.w3.org/TR/xml/>

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). Distributed systems: Concepts and design (5.^a ed.). Addison-Wesley.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Tesis doctoral, University of California, Irvine]. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>

OpenJS Foundation. (2024). Node.js documentation. <https://nodejs.org/en/docs>

Tanenbaum, A. S., & Van Steen, M. (2017). Distributed systems: Principles and paradigms (3.^a ed.). Pearson.

World Wide Web Consortium. (2001). Web Services Description Language (WSDL) 1.1.
<https://www.w3.org/TR/wsdl>

World Wide Web Consortium. (2007). SOAP Version 1.2 Part 0: Primer (2.^a ed.).
<https://www.w3.org/TR/soap12-part0/>